research_NeuroDect

Comprehensive Architectural Blueprint and Implementation Strategy for the NeuroDect Multimodal AI Screening System

The landscape of neurological diagnostics is fundamentally constrained by delayed detection methodologies and exceedingly high barriers to entry. Early warning signs of acute events such as Stroke and Bell's Palsy, alongside chronic degenerative conditions like Parkinson's Disease and Essential Tremor, manifest as subtle physiological anomalies—micro-tremors, marginal facial asymmetry, and vocal cadence degradation—that are frequently imperceptible to the untrained eye and often missed until the disease pathology has advanced significantly. Traditional neurological screening necessitates specialized clinical equipment, specialist consultations, and prolonged hospital visits, drastically reducing the temporal window for successful therapeutic intervention. The NeuroDect platform represents a conceptual paradigm shift: an accessible, browser-based screening tool designed to transform any standard consumer laptop into an early-warning neurological checkpoint.

Constructing a multimodal artificial intelligence inference engine within a severely constrained 36-hour hackathon environment requires ruthless prioritization, aggressive latency optimization, and the intelligent deployment of edge-computing paradigms. The system must operate seamlessly on low-end consumer hardware utilizing entirely free toolchains, integrating computer vision, acoustic analysis, and neuromotor interaction data into a unified Next.js and FastAPI architecture. The following exhaustive analysis delineates the optimal architectural blueprint, mathematical processing pipelines, implementation strategies, and user experience methodologies required to successfully deploy the NeuroDect prototype.

End-to-End Technical Architecture and Real-Time Data Flow

The foundational architecture of the NeuroDect system must carefully balance the computational intensity of continuous multimodal inference against the limitations of consumer-grade web browsers executing on potentially throttled Central Processing Units (CPUs). A hybrid, decentralized processing topology is strictly required to satisfy these conflicting parameters.

The architectural topology is segregated into a heavy-edge presentation layer and a highly optimized analytical backend. The presentation layer is constructed utilizing the React-based Next.js framework. Crucially, the system must not transmit raw video frames to the backend infrastructure. Streaming raw high-definition video over standard internet connections introduces crippling latency, saturates bandwidth, and fundamentally destroys the temporal resolution required for accurate micro-tremor frequency analysis. Instead, the Next.js client is responsible for executing all computer vision inference entirely on-device. By leveraging

MediaPipe's WebAssembly (WASM) binaries and WebGL acceleration, the client browser ingests the raw webcam feed, processes it through lightweight neural networks, and generates high-density numerical feature vectors representing spatial coordinates.

These extracted numerical coordinates—amounting to mere kilobytes of data per second rather than megabytes of raw video—are then transmitted to the analytical backend. The backend is architected utilizing FastAPI, selected specifically for its asynchronous event loop handling and high-throughput capabilities. The FastAPI server is tasked with executing complex statistical signal processing algorithms, such as Fast Fourier Transforms (FFT) for tremor analysis, and running secondary machine learning classifiers for acoustic processing. Following the analysis, the risk aggregation engine synthesizes the multimodal data and persists the results to Neon, a serverless PostgreSQL database chosen for its rapid provisioning and scalable architecture suitable for hackathons.

The real-time data flow relies on specific transport protocols tailored to the frequency of the incoming data.

Data Modality	Characteristic	Recommended Transport Protocol	Technical Justification
Facial & Hand Landmarks	Continuous, High-Frequency (15-30 Hz)	**WebSockets**	Eliminates HTTP header overhead associated with RESTful polling. Provides a persistent, bidirectional, low-latency communication channel critical for streaming spatial coordinates in real-time.
Acoustic Recordings	Discrete, Batch Payload (5-10 second files)	**REST (HTTP POST)**	Audio data is naturally captured in discrete blocks via the MediaRecorder API. A standard multipart/form-data POST request is highly efficient for binary blob transmission and simplifies backend routing.
Neuromotor Latency Metrics	Discrete, Low-Frequency (Single events)	**REST (HTTP POST)**	Reaction time data consists of simple JSON key-value pairs generated intermittently. The overhead of REST is negligible for this modality, simplifying state management.

Code snippet

```
graph TD
    subgraph Client Application
        Camera --> MP_Face
        Camera --> MP_Hand
        Mic[Microphone] --> AudioRec
        Mouse --> ReactTest

        MP_Face -- Serialized Landmark Coordinates --> WS_Client
        MP_Hand -- Serialized Landmark Coordinates --> WS_Client
    end

    subgraph Analytical Backend [FastAPI / Python]
        WS_Client -- JSON Coordinate Stream --> WS_Server
        AudioRec -- WAV/Blob HTTP POST --> REST_Audio[Acoustic Processing
Endpoint]
        ReactTest -- JSON HTTP POST --> REST_React[Interaction Metrics
Endpoint]

        WS_Server --> Tremor
        WS_Server --> Asym[Geometric Asymmetry Calculation]
        REST_Audio --> Librosa[Librosa Feature Extraction]
        Librosa --> XGBoost

        Tremor --> RiskAgg
        Asym --> RiskAgg
        XGBoost --> RiskAgg
        REST_React --> RiskAgg
    end

    subgraph Persistence and Explainability
        RiskAgg --> DB
        RiskAgg --> XAI[Explainable AI Heatmap Generator]
    end

    XAI -- Diagnostic Results & Heatmaps --> Client Application
```

Hardware Constraints and Dynamic Quality Scaling

Designing medical screening prototypes for low-end consumer devices requires anticipating hardware bottlenecks, specifically CPU exhaustion, limited Random Access Memory (RAM), and poor optical sensors. Traditional computer vision pipelines process an intensive detection algorithm on every single frame, which rapidly induces thermal throttling on consumer laptops.

MediaPipe fundamentally circumvents this by separating detection from tracking. The system executes a heavy detection model to locate the face or hand initially, and subsequently relies on a highly efficient, lightweight tracking model to follow the targets across subsequent frames. If confidence drops below a specified threshold, the heavy detection model is re-

invoked. Furthermore, MediaPipe automatically augments its inference calculator by utilizing the XNNPack Machine Learning Inference Library, which provides significant CPU acceleration when WebGL compute shaders are unavailable or unsupported by the browser.

Despite these inherent optimizations, the Next.js frontend must implement dynamic quality scaling to ensure the application remains responsive. A programmatic feedback loop must be established to monitor the delta time between `requestAnimationFrame` executions. If the client's rendering frame rate drops below a sustained threshold of 15 frames per second (FPS), the application must automatically trigger a series of degradation protocols.

First, the system should implement aggressive resolution downsampling. High-resolution video input does not linearly correlate with superior landmark detection accuracy, as MediaPipe internally normalizes and resizes frames prior to inference. By utilizing the `MediaStreamTrack.applyConstraints()` method, the application can dynamically reduce the webcam resolution from 1080p or 720p down to 640x480 or 320x240 pixels. This drastically reduces the memory footprint and the number of pixels the WASM binary must process, preserving critical CPU cycles without meaningfully degrading the topological accuracy of the landmark mesh.

Second, the system must dynamically adjust the model complexity. The MediaPipe initialization parameters allow developers to define the `model_complexity` integer. The application should initialize with a complexity of `1` (standard accuracy), but programmatically downgrade to `0` (a lighter, slightly less accurate neural network) if the hardware struggles to maintain real-time tracking.

Finally, deliberate frame dropping should be employed. Neurological tracking algorithms do not necessitate 60 FPS video feeds. Limiting the active inference loop to process every third or fourth frame ensures the system maintains an effective and stable 10 to 15 FPS. This temporal resolution is more than sufficient for accurately detecting the frequency of physiological micro-tremors and calculating structural facial asymmetry, while simultaneously preventing browser crashes on low-end hardware.

Latency Optimization for Real-Time Inference

Achieving near real-time feedback requires meticulous optimization of the FastAPI backend to prevent asynchronous blocking. Python's `asyncio` framework is immensely powerful for handling concurrent network requests, but mixing asynchronous input/output operations with blocking, CPU-bound mathematical tasks fundamentally destroys the event loop, resulting in catastrophic latency spikes for all connected clients.

Calculating Fast Fourier Transforms for tremor detection and extracting acoustic features using Librosa are highly CPU-intensive, blocking operations. To resolve this, the FastAPI architecture must offload these specific computations. The advanced implementation pattern involves utilizing the `concurrent.futures.ProcessPoolExecutor`. When the WebSocket endpoint receives an array of coordinates requiring frequency analysis, the endpoint utilizes the `asyncio.get_event_loop().run_in_executor()` method. This delegates the heavy

mathematical lifting to a separate worker process, allowing the primary FastAPI event loop to remain entirely unblocked and instantly responsive to incoming WebSocket packets or HTTP requests.

Furthermore, for tasks that do not require an immediate response to the user—such as persisting the aggregated diagnostic results to the Neon database or triggering an automated email report—the system must utilize FastAPI's native `BackgroundTasks` class. By declaring a background task parameter in the endpoint definition, the application can immediately return a success response to the Next.js client, while the database writing or external API communication occurs concurrently in a separate thread. This architectural pattern shaves critical milliseconds off the user-facing latency, creating a highly responsive and fluid screening experience.

Multimodal Diagnostic Pipelines: Visual, Audio, and Cognitive

The diagnostic efficacy of the NeuroDect platform relies entirely on the successful implementation of its four interactive screening tests, each requiring precise data extraction methodologies and specialized mathematical interpretations.

Visual Pipeline: Facial Asymmetry Detection (The Smile Test)

Facial asymmetry is a primary clinical diagnostic marker for identifying acute neurological events, specifically Stroke and Bell’s Palsy, which typically cause unilateral muscular paralysis or weakness. To quantify this symptomology objectively, the system employs the MediaPipe Face Landmarker, a sophisticated neural network that maps the human face using a dense topographical mesh of 478 distinct, three-dimensional coordinates.

The diagnostic implementation requires a two-step temporal process. Initially, the user is instructed to maintain a neutral facial expression while facing the camera. The system captures the spatial coordinates over a two-second interval, calculating the temporal average for each landmark to establish a stable, individualized geometric baseline. Subsequently, the system prompts the user to perform a dynamic expression, such as a wide smile. The application then isolates specific regions of interest, primarily focusing on the periocular regions (the muscles surrounding the eyes) and the oral commissures (the corners of the mouth).

The mathematical formulation for the asymmetry index requires calculating the Euclidean displacement vectors of corresponding bilateral landmarks between the neutral baseline and the dynamic expression. Because users sit at varying distances from the webcam, the raw pixel coordinates must be scale-normalized. This is achieved by calculating the interocular distance—the fixed distance between the outer corners of the left and right eyes—and utilizing it as a normalization denominator.

Let the spatial coordinate of a landmark on the left hemisphere of the face be denoted as $p_{L,i}$, and its corresponding bilateral landmark on the right hemisphere be $p_{R,i}$. The normalized

displacement magnitude for the i-th landmark pair during the expression phase is calculated as:

$$\Delta_i = \frac{\|p_{L,i}^{(expr)} - p_{L,i}^{(neutral)}\|_2 - \|p_{R,i}^{(expr)} - p_{R,i}^{(neutral)}\|_2}{D_{IO}}$$

Where D_{IO} represents the normalized interocular distance. An asymmetry score perfectly balanced near zero implies healthy, symmetrical muscular contraction. A score that significantly deviates from zero indicates unilateral muscular weakness, triggering a risk flag for potential acute neurological distress.

Visual Pipeline: Tremor Detection (The Hand Stability Test)

Resting and postural tremors are hallmark motor symptoms of chronic neurological disorders, particularly Parkinson's Disease and Essential Tremor. The clinical assessment of these involuntary oscillations can be digitized utilizing the MediaPipe Hand Landmarker, which reliably extracts 21 discrete joint coordinates per hand in three-dimensional space.

The user is instructed to extend their arm and hold their hand steady in front of the optical sensor. The client application records the spatial coordinates of highly expressive joints, specifically the index finger tip (Landmark 8) and the wrist joint (Landmark 0), capturing this telemetry over a sustained 5 to 10-second temporal window. This raw time-series coordinate data is continuously streamed to the FastAPI backend via WebSockets.

Raw optical tracking data is inherently noisy, often containing low-frequency voluntary movements (such as postural drift) and high-frequency camera jitter. To isolate the genuine physiological tremor, the Python backend applies a Savitzky-Golay filter, a digital smoothing polynomial filter that aggressively reduces high-frequency noise without distorting the underlying signal amplitude or frequency.

Once the signal is smoothed, the algorithmic focus shifts from the time domain to the frequency domain. Tremors are medically defined by their oscillatory frequency. The backend calculates the Power Spectral Density (PSD) of the velocity components utilizing Welch's method or a standard Fast Fourier Transform (FFT) implemented via the SciPy library.

A peak power detection algorithm scans the resulting frequency spectrum to identify significant oscillatory concentrations. Physiological tremors reside within well-documented frequency bands. Parkinsonian resting tremors typically present as distinct peaks in the 4 to 6 Hz range, whereas Essential Tremors generally manifest in the broader 4 to 10 Hz bandwidth. If the algorithmic analysis detects a sharp, symmetric power peak within these specific frequency bands, and the peak's amplitude exceeds a normative baseline threshold, the system flags the presence of a pathological tremor.

Audio Pipeline: Vocal Acoustic Analysis (The Speech Test)

Neurological degradation frequently impacts the complex motor coordination required for speech production, resulting in dysarthria (slurring of words), hypophonia (abnormally reduced vocal volume), and dysphonia (generalized vocal impairment). Acoustic analysis serves as a highly sensitive, non-invasive biomarker for these conditions.

The screening process begins with the browser's native MediaRecorder API capturing a 5 to 10-second acoustic sample. The user is instructed to sustain a continuous vowel sound or read a phonetically balanced sentence. This audio blob is transmitted via an HTTP POST request to the FastAPI backend, where the powerful Python library `librosa` is employed to extract sophisticated acoustic features.

The diagnostic algorithm focuses on three primary acoustic biomarkers:

1. **Jitter:** This metric quantifies the cycle-to-cycle variation in the fundamental frequency (F_0) of the voice. Elevated jitter indicates a systemic lack of neuromotor control over vocal cord vibration, a common early symptom in Parkinson's pathology.
2. **Shimmer:** This metric measures the cycle-to-cycle variation in the peak amplitude of the sound waves. High shimmer correlates heavily with vocal breathiness, instability, and uncontrolled noise emission during phonation.
3. **Harmonics-to-Noise Ratio (HNR):** This feature quantifies the ratio of periodic, harmonic energy to aperiodic, chaotic noise energy within the vocal signal. Calculated algorithmically as $10 \log_{10}(\frac{\text{Harmonic Energy}}{\text{Noise Energy}})$, a significantly lowered HNR suggests vocal cord incomplete closure or neurological hoarseness.

To translate these raw acoustic features into a diagnostic probability, the system utilizes a machine learning classifier. Developing a custom deep learning model for audio processing within a 36-hour hackathon is highly unadvisable due to data scarcity and training time constraints. Instead, the optimal implementation relies on an XGBoost classifier or a K-Nearest Neighbors (KNN) model. These classic machine learning architectures can be rapidly trained prior to the event using publicly available datasets, such as the UCI Parkinson's Disease classification dataset, which maps extracted acoustic features directly to clinical diagnoses. Evaluated implementations demonstrate that XGBoost classifiers achieve exceptional accuracy rates—upwards of 98% in controlled validation sets—while offering inference times measured in milliseconds, making it ideal for the real-time NeuroDect pipeline.

Cognitive Pipeline: Neuromotor Latency (The Reaction Test)

Cognitive processing speed and the subsequent neuromotor execution required to act upon stimuli often degrade significantly in the presence of acute concussions or chronic neurological conditions.

The latency assessment is executed entirely on the Next.js frontend, minimizing external network variables. Utilizing React state management, the application presents a stark visual stimulus—such as a large red geometric square that abruptly transitions to green after a

randomized temporal delay ranging between 1000 and 3000 milliseconds. The system programmatically records the delta in milliseconds between the rendering of the green stimulus and the detection of the user's `onClick` or `onKeyDown` event.

Healthy baseline reaction times for adult demographics average approximately 250 milliseconds. To ensure statistical validity and filter out anomalous preemptive clicks or user distractions, the system mandates five sequential trials. The absolute latencies and standard deviations are aggregated, and significant deviations from the 250-millisecond baseline are factored into the comprehensive neurological risk score.

Pretrained Models versus Custom Training Strategies

The severe temporal constraints of a 36-hour hackathon necessitate highly pragmatic decisions regarding machine learning implementations. A common pitfall is the attempt to train custom convolutional neural networks (CNNs) for facial landmark detection or recurrent neural networks (RNNs) for sequential tremor analysis. Such endeavors inevitably consume vast amounts of time in data curation, hyperparameter tuning, and troubleshooting vanishing gradients, often resulting in models that fail to converge or perform poorly on low-end hardware.

The NeuroDect architecture explicitly rejects custom deep learning model training in favor of integrating robust, pretrained, state-of-the-art models. Google’s MediaPipe ecosystem provides heavily optimized, quantized models that have been trained on immensely diverse, massive-scale datasets to identify 478 facial landmarks and 21 hand joints with sub-pixel accuracy. Attempting to replicate this accuracy with a custom model within the time limit is an exercise in futility. The engineering effort must instead be allocated toward the algorithmic interpretation of the data outputted by these pretrained models—specifically, writing the mathematical logic for the asymmetry formulas and the signal processing filters for tremor isolation.

The only exception where lightweight custom training is acceptable is the acoustic XGBoost classifier, strictly because the tabular dataset (e.g., the UCI Parkinson's dataset) is readily available, cleanly structured, and the training execution time for gradient boosting algorithms is measured in seconds rather than hours.

Usage Pattern Design and Signal Detection

The user experience (UX) architecture of a medical screening tool heavily influences the quality of the data collected. The manifestation of neurological degradation is rarely linear; it is characterized by subtle, episodic fluctuations. Consequently, the temporal frequency of application usage dictates the efficacy of the diagnostic algorithms.

Clinical neurological assessments traditionally rely on occasional, extensive testing sessions conducted in hospital environments. However, the primary advantage of at-home digital biomarkers is the ability to collect high-resolution longitudinal data. Applying Signal Detection Theory (SDT) principles, the UX must carefully balance the desire for granular data against

the risk of user fatigue. Mandating frequent, short sessions (e.g., a daily two-minute compulsory screening) yields exceptional longitudinal data, allowing the algorithms to detect microscopic deviations in baseline tremor frequencies over time. However, enforcing daily interaction rapidly induces severe user fatigue, friction, and eventual application abandonment. Conversely, encouraging only occasional use (e.g., once every six months) introduces massive data gaps, rendering the detection of early-stage chronic decline mathematically impossible.

The recommended UX paradigm is the "Weekly Comprehensive Check-in" supplemented by "Ad-Hoc Micro-Interactions." The user interface should proactively encourage the completion of the full four-step screening pipeline once a week to establish a reliable, high-fidelity trendline. To maintain engagement between these weekly sessions, the dashboard should allow users to perform single, 15-second micro-assessments—such as an isolated reaction time test or a rapid hand stability check—without the friction of executing the entire diagnostic suite.

Medical Dashboard and Explainable AI (XAI)

The design of the medical dashboard must prioritize absolute clarity, foster user trust, and deliver actionable insights without overclaiming clinical diagnostic authority. The visual representation of multi-test risk scoring must be instantly comprehensible to both lay users and potential medical professionals reviewing the data.

To accomplish this within the Next.js framework, selecting the optimal charting library is a critical architectural decision. The library must be lightweight, deeply integrated with React component lifecycles, and highly customizable.

Charting Library	Rendering Engine	Architectural Fit	Assessment for NeuroDect
Chart.js (react-chartjs-2)	Canvas	Imperative	Highly performant for massive datasets, but the Canvas API is less idiomatic in a declarative React environment. Styling can be cumbersome.
D3.js	SVG	Low-Level	Offers limitless power and customization, but possesses a notoriously brutal learning curve that is entirely unsuited for rapid hackathon deployment.
Recharts	SVG	Declarative Components	**Optimal Selection.** Built atop D3 but exposes intuitive, composable React components (e.g., `<LineChart>`, `<Tooltip>`). It is incredibly fast to implement and highly responsive.

The dashboard should utilize Recharts to render longitudinal line graphs that chart the composite "Neurological Risk Score" over time, empowering users to visually identify upward trending risk trajectories.

Furthermore, the implementation of Explainable Artificial Intelligence (XAI) is paramount. In the medical technology domain, opaque "black box" models systematically destroy clinical trust. If the NeuroDect system flags a high risk for facial asymmetry, it must explicitly visualize *why* that determination was made.

Instead of presenting an abstract risk percentage, the system must generate visual saliency maps or diagnostic heatmaps. Upon completing the Smile Test, the application captures a still frame of the user's face and overlays the MediaPipe Face Mesh onto an HTML Canvas element. The application iterates through the calculated asymmetry indices of all 478 landmarks. Utilizing a dynamic color gradient function, landmarks demonstrating asymmetry deviations below the normative threshold are rendered in a translucent green, while landmarks exceeding the threshold shift drastically toward a highly visible, opaque red. Rendering this dynamic, localized heatmap instantly communicates to the user that the model is flagging a specific structural anomaly—such as a drooping left labial commissure—thereby bridging the critical gap between complex algorithmic output and intuitive human interpretability.

Advanced Integrations: WebRTC, Geolocation, and Automated Reporting

To elevate the NeuroDect prototype from a localized diagnostic toy to a comprehensive, interconnected healthcare platform, several external APIs and advanced networking protocols must be integrated.

QR-Based Mobile Camera Streaming (WebRTC)

Laptop webcams are historically of poor optical quality and are physically cumbersome to position correctly for precise tremor tracking. Enabling the user to utilize their high-resolution smartphone camera as the primary optical input device provides a massive usability and data-quality advantage.

Implementing a traditional signaling server for WebRTC using Socket.io or dedicated STUN/TURN servers within 36 hours is fraught with complexity and debugging risks. An innovative alternative is the implementation of a QR-based WebRTC signaling channel.

The workflow operates entirely peer-to-peer: The Next.js desktop application generates a WebRTC Session Description Protocol (SDP) "offer" containing the necessary cryptographic keys and ICE network candidates, encoding this massive text string into a high-density QR code rendered directly on the desktop monitor. The user scans this QR code utilizing their smartphone's native camera. This action redirects the smartphone to a lightweight mobile webpage hosted by the Next.js application, which decodes the SDP offer, generates an SDP "answer", and renders a second QR code on the mobile screen. The user holds the

smartphone screen up to the desktop webcam, which scans the answer code, completing the manual, optical handshake. Upon completion, a direct peer-to-peer WebRTC video stream is established, transmitting the mobile camera feed to the desktop browser without intermediate signaling servers.

Note on Architectural Risk: While this QR signaling approach brilliantly eliminates backend dependencies, it is highly susceptible to ICE candidate negotiation failures on restrictive, symmetric NAT networks (such as typical university hackathon Wi-Fi). A robust implementation must include a graceful fallback mechanism, reverting to the standard laptop webcam if the P2P connection fails to materialize.

Hospital Locator and Automated Reporting APIs

Upon detecting severe acute anomalies—such as pronounced facial drooping highly indicative of an active Stroke—the system must immediately pivot from screening to emergency facilitation.

The Geoapify Places API serves as the optimal integration choice, offering an expansive free tier of 3,000 requests per day without requiring credit card authentication. The integration relies on the HTML5 Geolocation API (`navigator.geolocation.getCurrentPosition()`) to acquire the user's precise latitudinal and longitudinal coordinates. The Next.js frontend then executes a REST call to the Geoapify endpoint: `https://api.geoapify.com/v2/places?categories=healthcare.hospital&filter=circle:{lon},{lat},5000&apiKey={API_KEY}`. The application parses the returned JSON payload and dynamically populates the dashboard with the nearest medical facilities, emergency contact numbers, and routing distances.

To ensure patient compliance with the longitudinal monitoring paradigm, automated reminders must be dispatched. The Gmail API provides a robust, zero-cost mechanism for programmatic email dispatch. Following the configuration of a Google Cloud Project and the generation of OAuth2 credentials, the FastAPI backend integrates the `google-api-python-client` library. Utilizing the aforementioned `fastapi.BackgroundTasks` architecture, the backend constructs a MIME multipart message and triggers the `gmail.users().messages().send()` method as an asynchronous background thread, ensuring the notification dispatch does not induce latency in the primary application event loop.

Step-by-Step 36-Hour Implementation Strategy

In a hackathon setting, the probability of failure increases exponentially with scope creep. A rigidly defined, chronological build plan is mandatory.

Project Phase	Timeline	Critical Actions	Deliberate Exclusions (Shortcuts)
Phase 1: Foundational Infrastructure	Hours 0 - 6	Scaffold Next.js and FastAPI. Establish WebSocket protocols and JSON data schemas. Provision Neon Database.	Do not build complex OAuth/JWT user authentication. Mock user sessions using a UUID stored in `localStorage` to save debugging hours.
Phase 2: Core Computer Vision	Hours 6 - 16	Integrate `@mediapipe/face_mesh` and `@mediapipe/hands`. Construct the `requestAnimationFrame` loop on the HTML `<canvas>`. Write the Euclidean asymmetry functions.	Do not calculate FFTs in JavaScript on the frontend. Transmit the raw 3D coordinate arrays directly to the FastAPI backend.
Phase 3: Acoustic & Signal Processing	Hours 16 - 24	Implement `MediaRecorder` API. Integrate `librosa` on the backend for Jitter, Shimmer, and HNR extraction. Implement `scipy.signal.welch` for tremor FFT.	Use a pre-trained `xgboost` model for audio classification, loaded via `joblib`. Do not waste time tuning custom hyperparameters.
Phase 4: Integrations & Visualization	Hours 24 - 32	Construct Recharts dashboard. Implement the XAI Heatmap overlay logic. Integrate Geoapify and Gmail APIs.	Build the QR WebRTC signaling if time permits; otherwise, strictly utilize the laptop webcam.
Phase 5: Polish & Presentation	Hours 32 - 36	Optimize UX responsiveness. Ensure graceful error handling for missing hardware permissions. Generate synthetic historical database entries.	Exclude extensive cross-browser compatibility testing. Optimize exclusively for Google Chrome.

Strategic Trade-Offs and Architectural Decisions

Developing sophisticated artificial intelligence systems under extreme temporal constraints requires calculated architectural compromises. The following explicit trade-offs must be embraced and actively defended during the project presentation:

Decision Parameter	Chosen Architecture	Discarded Alternative	Technical Justification
Inference Location	**Client-Side WASM (MediaPipe)**	Server-Side Python (OpenCV)	Server-side computer vision requires streaming raw video frames, fundamentally violating the "low-end hardware" constraint due to massive network latency, bandwidth saturation, and immense backend compute costs.
Model Architecture	**Pretrained Toolchains**	Custom Deep Learning Training	Training custom neural networks requires curated datasets and graphical compute time entirely absent in a 36-hour window. Ensembling pretrained tools yields high accuracy instantly.
Telemetry Protocol	**Persistent WebSockets**	RESTful Polling	REST introduces severe HTTP header overhead for continuous coordinate streaming. WebSockets provide the persistent, low-latency, bidirectional channels necessary for live telemetry.
Clinical Validation	**Synthetic Threshold Approximation**	Rigorous Clinical Trials	Explicitly excluded from the scope. The system is presented as a preliminary *screening tool*, not a definitive diagnostic device. Thresholds for tremors and asymmetry are approximated based on medical literature to demonstrate functional capabilities.

Premium Hackathon Edge and Presentation Optimization

To transcend basic functionality and excel in a highly competitive evaluation environment, the implementation must deliver exceptional presentation value. The inclusion of "Wow Factor" features is critical. Rendering the dynamic, color-coded XAI 3D facial mesh that visually highlights muscular asymmetry provides immediate, stunning visual feedback that judges can intuitively comprehend without requiring a background in neurology. Furthermore, successfully demonstrating the QR-based WebRTC mobile-to-desktop camera handoff proves advanced architectural networking competence and solves a tangible real-world usability issue.

Crucially, the live demonstration must be flawless. Developers must not rely on the judges possessing actual neurological disorders to trigger the system's positive detection pathways. The application must include a hidden "Demo Mode" toggle. When activated, this mode should artificially inject a simulated 5 Hz mathematical oscillation into the incoming hand

tracking coordinate stream, and deliberately skew the facial landmark matrix to simulate unilateral paralysis.

Finally, the presentation must align heavily with the broader socio-economic judging criteria. The database must be pre-populated with 30 days of synthetic longitudinal data for a fictitious patient, allowing the Recharts dashboard to instantly display compelling trend lines that demonstrate a simulated cognitive decline. By emphasizing that the architecture relies entirely on free toolchains, serverless scaling, and client-side processing, the system costs virtually nothing to operate per user. This effectively democratizes access to sophisticated neurological screening, directly addressing the core systemic problem of delayed diagnostic detection in under-resourced, remote, and vulnerable global populations.